Bacterial AMR project: what we are doing, from zero

Study guide · M. Dyen Asif (24I-0608), FAST-NUCES Islamabad · 1 Oct 2026 · matches the code in **AMR_AL/** (config.py) exactly

The project in three sentences. A computer model can predict from a bacterium's MALDI-TOF mass spectrum whether an antibiotic will still work, a day before the lab result. Such models get worse over time and at new hospitals, so they must be retrained with fresh lab results, and every lab result costs a real test. **We study which new isolates a lab should send for testing so the retrained model recovers the most accuracy for the fewest tests** (this is called *active learning*).

What exists today

Item	State
Plan, literature (novelty) check, Colab guide, your slides	done: AMR_Plan.pdf, AMR_Literature_Check.pdf, AMR_Colab_Setup_Guide.pdf, AMR_Active_Learning_Proposal.pptx
Code package **amr_al** + notebooks 00–04	written; self-test 17/17 on synthetic data; private repo github.com/DyneStein/amr-active-learning
Real DRIAMS download, real GPU runs	**not yet done** (next step: notebooks 00 → 01 → 02)
Formal novelty search in Scopus / PubMed (FAST library)	**not yet done**; needed before writing "not done before"

Contents

1 The problem (biology and why models decay) · 2 The idea: the active-learning loop · 3 Research questions and baselines · 4 The data: DRIAMS, features, labels · 5 Train / pool / test splits · 6 The model: XGBoost and tuning · 7 The decision threshold · 8 Selection strategies · 9 Conformal prediction and safety routing · 10 Pseudo-labels (semi-supervised) · 11 How results are measured · 12 The four experiments · 13 Randomness and the multi-seed design · 14 Ablation study · 15 How the code runs (Colab, workers) · 16 Risks and what to check first · 17 Key references

Every symbol is explained in a box right below the place it first appears. Numbers in the tables are the actual settings in **config.py**. Things that are only planned or suggested are labelled as such.

1 · The problem

Antimicrobial resistance (AMR)

Bacteria can evolve to survive antibiotics that used to kill them (random mutations under antibiotic pressure, and resistance genes passed between bacteria). A patient infected with a resistant strain does not improve on the standard drug. Bacterial AMR was directly responsible for ≈ 1.14 million deaths in 2021 (GBD 2021, as on your slide 4).

How a hospital lab tests resistance today

1. A sample (blood, urine, swab) is taken and grown on a plate until a pure colony of one organism is obtained: the **isolate**.
2. **Species identification** by MALDI-TOF (minutes): the lab now knows it is, say, *E. coli*.
3. **Antimicrobial susceptibility testing (AST)**: the isolate is grown with each antibiotic for 18–24 h; the result per antibiotic is **S** (susceptible: the drug works), **I** (intermediate: works only at higher dose / some sites), or **R** (resistant: the drug fails).

MALDI-TOF mass spectrometry

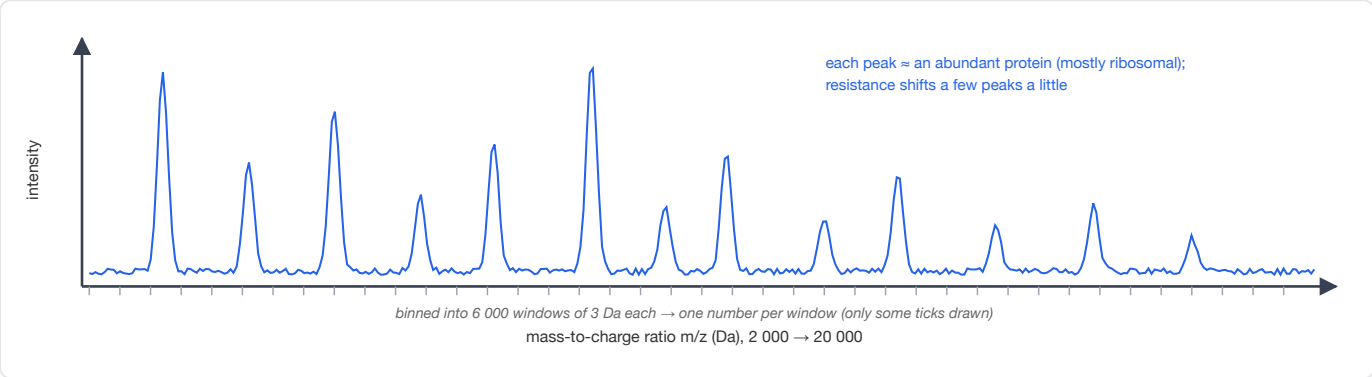


Figure 1. A (schematic) MALDI-TOF spectrum and the 3-Da binning that turns it into 6 000 numbers.

MALDI-TOF = matrix-assisted laser desorption/ionisation – time of flight. A bit of the colony is mixed with a chemical matrix and hit by a laser; its proteins fly through a tube, and lighter ones arrive first. The machine records how many ions arrive at each **mass-to-charge ratio m/z**: a **spectrum**, a curve with peaks. Labs already do this for every isolate (to name the species), so the spectrum is *free data*. Resistance mechanisms slightly change some proteins, which shifts or adds peaks; a machine-learning model can learn these patterns and **predict S/R about a day earlier** than AST.

Why the models decay, and why fixing them costs money

- **Temporal drift:** bacterial populations change (new strains spread), and instruments drift (cleaning, calibration). A model trained on 2015–16 is tested on later years. Weis et al. Nat Med 2022 **A** saw drops on DRIAMS when testing on a later year; Wiesmann et al. J Clin Microbiol 2025 **A** saw AUROC fall by 0.10–0.25 within 18 months in a German hospital.
- **Site shift:** a model built in Basel works worse in another hospital (different machines, patients, strains); Duroux et al. GigaScience 2026 **A** report large cross-site drops.
- **The fix is retraining with new lab results ("labels").** Computing is cheap; every label is a real AST test (reagents, 18–24 h, technician time). So the real question is **which isolates to test**.

Words used here

- **isolate** — one pure bacterial culture from one patient sample; the unit we predict for.
- **phenotype / phenotypic test** — the observed behaviour (does it grow with the drug?), measured by AST; the 'true label'.
- **Da (dalton)** — unit of molecular mass (≈ mass of one hydrogen atom). 2 000–20 000 Da covers most ribosomal proteins.
- **drift / shift** — the data a model sees after training differs from its training data, so it makes more mistakes.
- **label** — the answer we want the model to predict: here R (1) or S (0).

2 · The idea: the active-learning loop

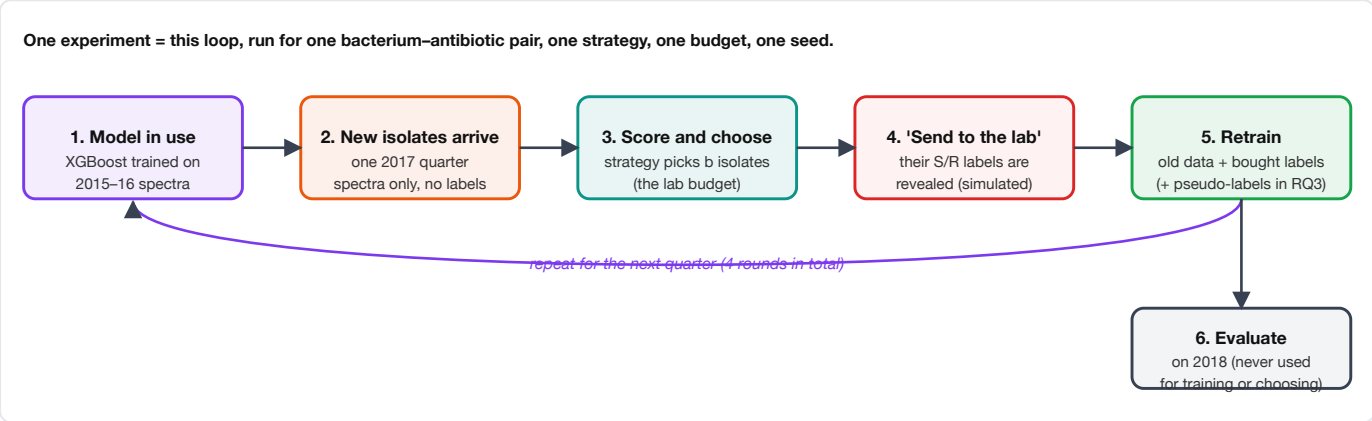


Figure 2. What one simulated experiment does, round by round.

Active learning = the model chooses which unlabelled examples should be labelled next, so that labels are spent where they help most. Our version is **pool-based, batch-mode, with a wet-lab oracle**:

- **Pool:** the isolates that arrive during 2017 (spectra known, labels hidden).
- **Batch / round:** isolates arrive in 4 batches (quarters). In each round the strategy may buy at most b labels from that batch, then the model is retrained once.
- **Oracle:** whoever provides true labels. In reality the lab; in our study a **simulation**: DRIAMS already contains every isolate's AST result, so "sending to the lab" just reveals the stored label. **Cost = number of labels revealed.** This is the standard way active learning is evaluated, and it is honest as long as a label is never used before it is "paid for".
- **Budget B:** total labels bought over the year (40, 100, 200, 400 or 800), spread evenly over the 4 rounds.

$$b_r = \lfloor B / R \rfloor + (1 \text{ if } r < B \bmod R \text{ else } 0), \quad r = 0, 1, \ldots, R-1 \tag{1}$$

where

B total budget for the whole pool period

R number of rounds (N_ROUNDS = 4 temporal; 1 cross-site)

b_r labels allowed in round r; e.g. B = 100 → 25, 25, 25, 25; B = 40 → 10 each

$\lfloor\ \rfloor$, mod round down; remainder after division

Why this could beat random testing: random testing spends most labels on isolates the model already gets right. A smart strategy spends them on isolates the model is unsure about (uncertainty), on unusual spectra it has never seen (diversity), or both (hybrid).

Words used here

- **pool-based** — the learner can see the whole current batch before choosing (vs. 'stream-based', deciding one by one).
- **query strategy** — the rule that picks which examples to label.
- **retrain** — fit the model again from scratch on old training data + all labels bought so far.

3 · Research questions and baselines

RQ	Question (as in AMR_Literature_Check.pdf)	Experiment
RQ1	Over a realistic year of arrivals, how many lab-tested isolates does each strategy need to recover 95% of what full retraining recovers?	E1_temporal
RQ2	Uncertainty vs spectral diversity vs hybrid, when isolates arrive in clusters of near-identical spectra.	E1_temporal
RQ3	At equal budget, does buying labels + using free pseudo-labels on the untested spectra beat either alone?	E2_semisup
RQ4	A model trained at Basel (DRIAMS-A) is deployed at hospital B, C or D: how few local tests close the gap?	E4_crosssite
RQ5	If the lab only tests isolates the model is unsure about (conformal safety routing), how good are those labels for retraining, and how many dangerous errors happen on the untested ones?	E3_routing

Baselines every result is compared with: none (never retrain: lower bound) · **full** (label every 2017 isolate: upper bound) · **random** selection at the same budget · **ssl_only** (pseudo-labels, no bought labels) · for RQ4 also zero-shot (Duroux 2026) from the literature. RQ1–RQ3 make one complete paper; RQ4 and RQ5 raise its level or form a second paper (supervisors decide).

4 · The data: DRIAMS

DRIAMS (Database of Resistance Information on Antimicrobials and MALDI-TOF Mass Spectra; Weis et al. Nat Med 2022 **A**) is public on Dryad (doi:10.5061/dryad.bzkh1899q, licence CC0). It has >300 000 spectra and >750 000 AST labels from four Swiss diagnostic labs:

Site	Years	Archive size	Used for
DRIAMS-A (University Hospital Basel)	2015, 2016, 2017, 2018	86 GB	everything (the only multi-year site)
DRIAMS-B	2018	3.7 GB	RQ4 target hospital
DRIAMS-C	2018	12.4 GB	RQ4 target hospital
DRIAMS-D	2018	42.6 GB	RQ4 target hospital

Notebook 01 streams each archive once and keeps only two things (≈ 2–4 GB for DRIAMS-A): the **pre-binned spectra** *<site>/binned_6000/<year>/<code>.txt* and the **label tables** *<site>/id/<year>/<year>_clean.csv*. They are saved as **X.npy** (one row per spectrum, float16, multiplied by 10^4) and **meta.csv** (same order: isolate *code*, *species*, one column per antibiotic with S/I/R, and *acquisition_date* if present). *The real folder layout inside the archive is not yet confirmed; notebook 01's "peek" checks it first.*

Features: what the model sees

$$x = (x_1, \ldots, x_{6000}), \quad x_k = \text{normalised intensity summed over m/z} \in [2000 + 3(k-1),\ 2000 + 3k)\ \text{Da} \tag{2}$$

where

- x the feature vector of one isolate: 6 000 non-negative numbers
- x_k bin k: total (normalised) signal in a 3-Da window
- ***normalised*** divided by the total ion current (sum of all intensities), so every spectrum sums to 1 and the overall brightness of a measurement does not matter; each bin is therefore ≈ 10^{-4}, hence the $\times 10^4$ before storing

The DRIAMS authors produced these vectors with a fixed pre-processing chain (as described in Weis et al. 2022): square-root variance stabilisation, Savitzky–Golay smoothing, SNIP baseline removal, total-ion-current normalisation, trimming to 2 000–20 000 Da, and 3-Da binning. We use their bins unchanged, so results are comparable with theirs.

Labels: what the model predicts

$$y = 1 \text{ if AST result} \in \{R, I\};\quad y = 0 \text{ if S};\quad \text{otherwise the isolate is dropped for that antibiotic} \tag{3}$$

"Intermediate counts as resistant" follows the DRIAMS authors' own code (maldi-learn) and is the clinically cautious choice: a doctor should not rely on a drug that only partly works.

The three species–antibiotic pairs (one separate model each)

Code	Species	Antibiotic	What resistance usually means
ecoli_cro	*Escherichia coli*	ceftriaxone (3rd-generation cephalosporin)	mostly ESBL enzymes (extended-spectrum β-lactamases) that destroy the drug
kpneu_cro	*Klebsiella pneumoniae*	ceftriaxone	ESBL / other β-lactamases; the pair that decayed most in published studies
saur_oxa	*Staphylococcus aureus*	oxacillin	MRSA: the *mecA* gene makes an altered penicillin-binding protein (PBP2a)

These are the three pairs studied in the DRIAMS paper. Resistant isolates are the **minority class** (the exact % per pair and year is printed by notebook 02); this imbalance is why AUPRC and very-major errors matter (Section 11).

Words used here

- **float16** — a number stored in 16 bits (half the memory of normal 32-bit floats); enough precision for spectra after $\times 10^4$ scaling.
- **species–antibiotic pair** — one prediction task, e.g. 'is this E. coli resistant to ceftriaxone?'
- **minority class** — the rarer answer (here R); a model can look accurate by always saying the common answer.

5 · Train / pool / test splits

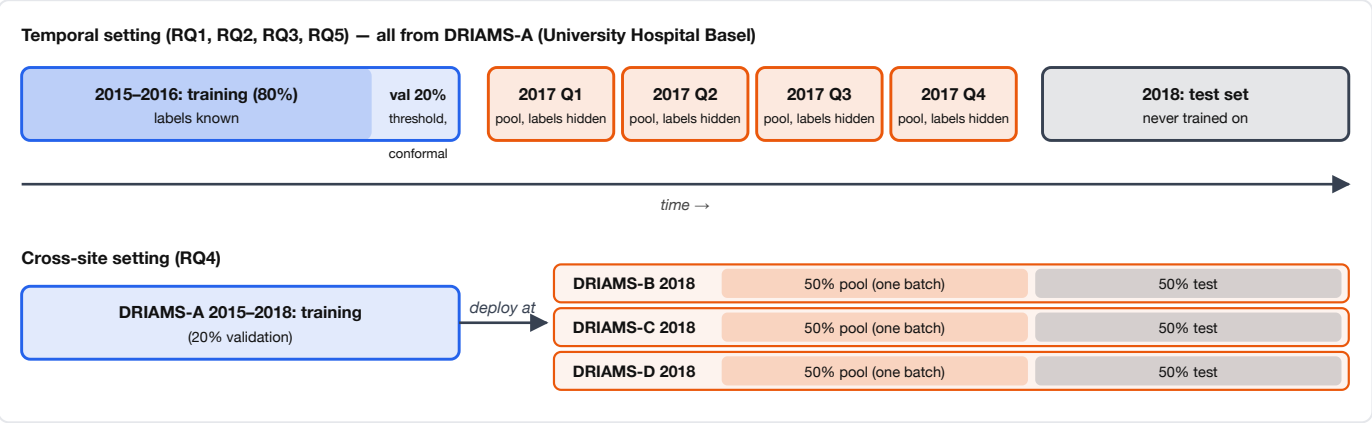


Figure 3. The two settings. Same data type (spectra + S/R) everywhere; only time or hospital changes.

Part	Exact definition in the code	Used for
Training set	DRIAMS-A 2015 + 2016 isolates of the pair, minus the validation part	fitting the starting model; always kept in later retraining
Validation set	20% of 2015–16, drawn **stratified** (20% of the R and 20% of the S separately) with the job's seed	choosing the decision threshold (Sec. 7) and calibrating conformal sets (Sec. 9). Never trained on.
Pool	all DRIAMS-A 2017 isolates of the pair, split into 4 rounds	the only place labels can be bought
Test set	all DRIAMS-A 2018 isolates of the pair	reporting results only
Cross-site (RQ4)	train on DRIAMS-A 2015–2018 (20% validation); target site 2018 split 50/50 stratified (seed + 1000) into pool (one round) and test	"how few local tests does a new hospital need?"

How the 2017 pool is cut into rounds

- If every 2017 isolate has an acquisition date: **calendar quarters** (Jan–Mar = round 1, …).
- If dates are missing: a fixed pseudo-random batch from a hash of the isolate code (the same for every seed and strategy). Notebook 02 reports which mode was used, and results record it.

Rules that prevent cheating (data leakage)

- The test year is never used for training, tuning, thresholds or choosing isolates.
- Hyper-parameters are tuned on 2015–16 only.
- A pool label is used only after the strategy has "bought" it; pseudo-labels never use the true label (their accuracy is recorded for analysis only).

Words used here

- **stratified** — split so that each part keeps the same share of R and S as the whole.
- **data leakage** — information from the test data sneaking into training or design choices, making results look better than they are.
- **hash** — a fixed scrambling function: the same code always lands in the same batch.

6 · The model: XGBoost

A **decision tree** asks yes/no questions about features (e.g. "is bin 1 247 above 0.8?") and ends in a leaf with a score. **Gradient boosting** builds many small trees one after another; each new tree corrects the mistakes of the trees so far. **XGBoost** Chen & Guestrin, KDD 2016 **A** is a fast, regularised implementation that also runs on a GPU (a T4 on Colab).

$$F(x) = \Sigma_{m=1\ldots M}\, \eta \cdot f_m(x), \qquad p(x) = \sigma(F(x)) = 1 / (1 + e^{-F(x)}) \tag{4}$$

where

- f_m tree number m (its leaf score for spectrum x)
- M number of trees (n_estimators)
- η learning rate: how much each tree is trusted
- σ the logistic (sigmoid) function, turning any score into a probability between 0 and 1
- $p(x)$ predicted probability that the isolate is resistant

Training minimises the **weighted log-loss** (how surprised the model is by the true labels) plus a penalty on large leaf scores:

$$L = -\,\Sigma_i\, w_i\, [\, y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i))\,] + \lambda\, \Sigma\, (\text{leaf scores})^2 \tag{5}$$

where

- w_i sample weight: 1 for original training data and bought labels (NEW_LABEL_WEIGHT = 1.0), 0.5 for pseudo-labels (SSL_WEIGHT)
- λ reg_lambda = 1.0, L2 regularisation

Setting (config.py)	Default	Tuning grid	Meaning
n_estimators	300	300, 600	number of trees M
max_depth	6	4, 6	questions per tree path (complexity)
learning_rate	0.1	0.05, 0.1	η in (4)
subsample	0.8	—	each tree sees a random 80% of the isolates
colsample_bytree	0.3	0.1, 0.3	each tree sees a random 30% of the 6 000 bins
min_child_weight, reg_lambda	1, 1.0	—	minimum data per leaf; L2 penalty

Tuning (notebook 02): all $2^4 = 16$ grid combinations, each scored by 3-fold stratified cross-validation AUROC on 2015–16 only; the best is saved in *best_params.json* and used by every job of that pair. **Alternative model:** logistic regression (MODEL = "lr": standardised features, C = 0.05, class-balanced), useful as an ablation (Sec. 14).

7 · The decision threshold

The model gives a probability p; the lab needs a yes/no. Missing a resistant isolate is dangerous, so the threshold t is set on the validation set so that at least 90% of truly resistant isolates are flagged (TARGET_SENSITIVITY = 0.90):

$$t = p_{(k)}^{R}, \quad k = \lceil 0.90 \cdot n_R \rceil; \quad \text{predict R if } p \geq t \tag{6}$$

where

- n_R number of resistant isolates in the validation set
- $p_{(k)}^{R}$ the k-th largest predicted probability among those resistant isolates: the largest cut-off that still catches 90% of them
- $\lceil\ \rceil$ round up

The threshold is recomputed after every retraining, as a lab would re-validate a model.

Words used here

- **cross-validation** — split the data into k parts; train on k−1, test on the last; rotate; average. Gives a fair score without touching the test set.
- **sensitivity (recall for R)** — share of truly resistant isolates the model calls resistant.
- **GPU** — graphics processor; trains XGBoost several times faster than a CPU.

8 · Selection strategies

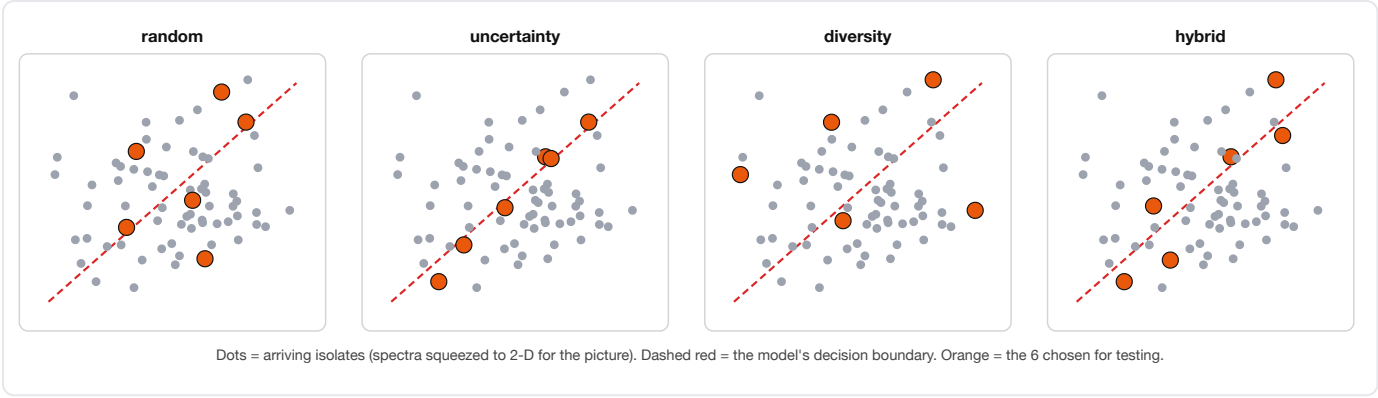


Figure 4. What each strategy tends to pick (illustration).

In each round the current model scores every arriving isolate (probability p_i); the strategy returns the b_r isolates to "test".

Strategy	Rule (as implemented)	Idea / weakness
random	b_r isolates uniformly at random	the fair reference every method must beat
uncertainty	the b_r isolates with the smallest $\lvert p_i - t \rvert$ (closest to the decision threshold)	learn where the model hesitates / may pick many near-identical isolates
diversity	greedy **k-center** (Sener & Savarese ICLR 2018 **A**) in a 50-dimensional PCA space: repeatedly take the isolate farthest from everything already labelled (eq. 7)	cover unusual / new spectra (new strains) / ignores the model's doubts
hybrid	take the 4·b_r most uncertain, cluster them with k-means into b_r clusters, test the isolate nearest each cluster centre	uncertain *and* varied; answers RQ2
routing	only isolates whose conformal set is ambiguous (Sec. 9); if more than b_r, a random subset; never filled up with others	a lab's safety rule used as a strategy (RQ5)
routing_all	every ambiguous isolate, no budget cap	what pure safety routing would cost

$$j^* = \operatorname{argmax}_{j \in \text{candidates}} \; \min_{\ell \in \text{labelled}} \lVert e_j - e_\ell \rVert, \quad \text{repeated } b_r \text{ times (labelled set grows each time)} \tag{7}$$

where

e_j isolate j's spectrum projected onto 50 principal components (PCA_DIM = 50), fitted once on the training years

$\lVert\ \rVert$ Euclidean distance

labelled the training set plus every pool isolate bought so far

Words used here

- **PCA (principal component analysis)** — rotates the 6 000-number spectra so the first few new axes carry most of the variation; keeping 50 axes makes distances fast and less noisy.
- **k-means** — groups points into k clusters around k centres, so each cluster holds similar spectra.
- **argmax / argmin** — the item that gives the largest / smallest value.

9 · Conformal prediction and safety routing (RQ5)

A probability like p = 0.55 does not say how much to trust it. **Conformal prediction** turns the model's outputs into a **prediction set** that contains the true label with a guaranteed probability (here ≥ 90%). We use the **class-conditional ("Mondrian") split** version, calibrated separately on validation isolates that are truly S and truly R, so the rare R class gets its own guarantee.

$$\text{score for class c:}\quad s = 1 - \hat{p}(c), \quad \hat{p}(R) = p, \quad \hat{p}(S) = 1 - p \tag{8}$$

$$q_c = \text{the } \lceil (n_c + 1)(1 - \alpha) \rceil \text{-th smallest score among the } n_c \text{ validation isolates of class c} \tag{9}$$

$$\text{set}(x) = \{\, c : 1 - \hat{p}_x(c) \le q_c \,\} \quad \Rightarrow \quad P(\text{ true class} \in \text{set}\,) \ge 1 - \alpha \tag{10}$$

where

- α allowed miss rate: CONFORMAL_ALPHA = 0.10 → 90% coverage per class
- n_c number of validation isolates whose true class is c
- q_c class-c cut-off: 90% of true-c validation isolates had a score at or below it
- $(n+1)$ small-sample correction that makes the guarantee exact

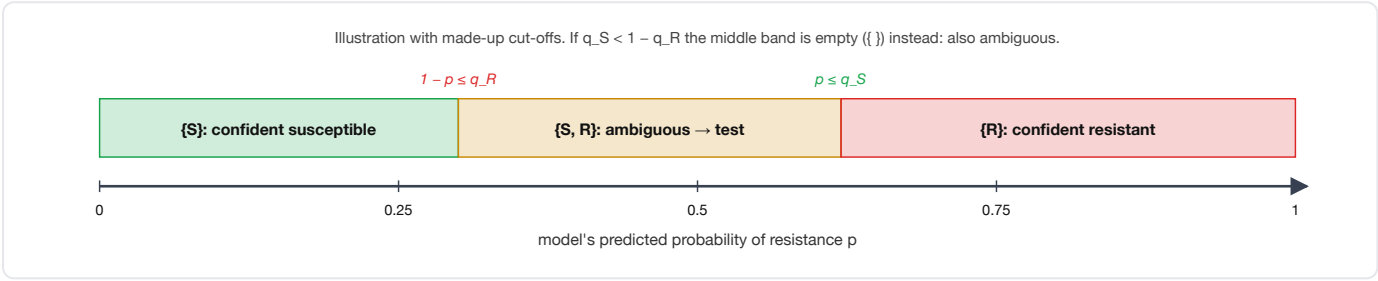


Figure 5. Prediction sets as a function of p. **Ambiguous** = set {S, R} or empty { } → the model cannot decide → send to the lab.

Honest caveat: the guarantee assumes new isolates resemble the validation isolates ("exchangeability"). Under drift this is exactly what fails, so coverage on 2018 will likely be below 90%; measuring that is part of RQ5.

10 · Pseudo-labels (semi-supervised, RQ3)

Untested 2017 spectra are free. In the "+ssl" strategies, after buying labels, every *untested* isolate with p ≥ 0.95 gets the pseudo-label R and every one with p ≤ 0.05 gets S; they join retraining with weight 0.5. **ssl_only** uses pseudo-labels with zero bought labels. The code records how often pseudo-labels were right (pseudo_acc), which reveals when the model confidently teaches itself mistakes.

11 · How results are measured

Everything is computed on the 2018 test set (or the target hospital's test half) after the final round, and also after every round.

	truly R	truly S
predicted R	TP (true positive)	FP: **major error** (a working drug is withheld)
predicted S	FN: **very major error** (a failing drug is given)	TN (true negative)

$$\text{VME} = \text{FN} / \#R, \quad \text{ME} = \text{FP} / \#S, \quad \text{balanced accuracy} = \tfrac{1}{2}\,(\text{TP}/\#R + \text{TN}/\#S) \tag{11}$$

$$\text{AUROC} = P(\text{ p of a random R isolate} > \text{p of a random S isolate}\,) \tag{12a}$$

$$\text{AUPRC} = \text{area under the curve of precision TP/(TP+FP) against recall TP}/\#R \tag{12b}$$

where

- $\#R, \#S$ numbers of truly resistant / susceptible test isolates
- $AUROC$ threshold-free ranking quality: 0.5 = coin flip, 1 = perfect
- $AUPRC$ average precision; its 'coin-flip' level equals the share of R, so it is much stricter than AUROC when R is rare

Clinical context: VME is the dangerous error. For commercial AST devices, regulators expect very low rates (FDA guidance: VME ≤ 1.5%, ME ≤ 3%); ML models are far from that, which is why the threshold favours sensitivity.

Errors on untested isolates (op_vme_count, op_me_count): in each round, the isolates *not* sent to the lab would be reported with the model's answer. Counting their VMEs measures the clinical price of each strategy during the

year, the key number for RQ5.

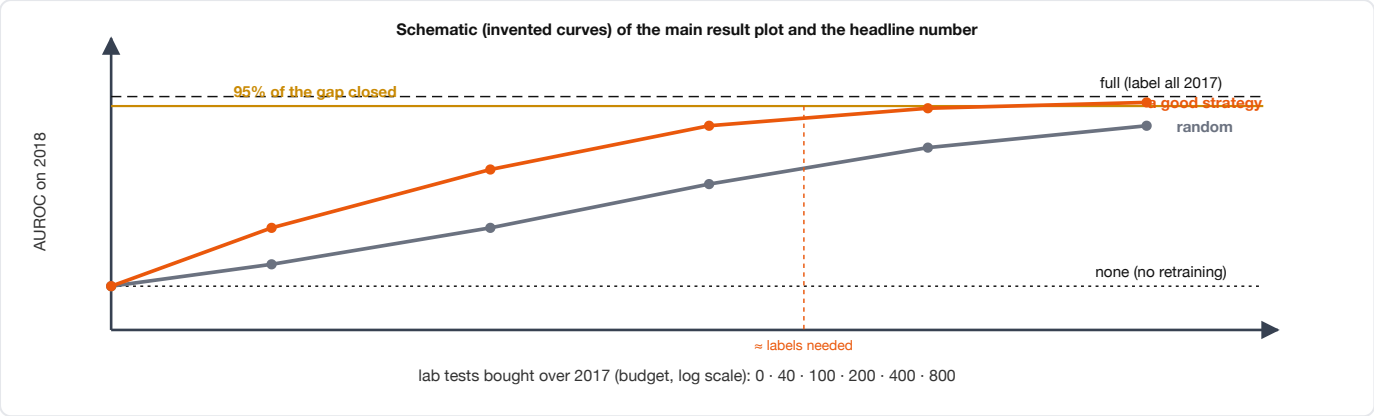


Figure 6. The main plot: final AUROC vs budget for each strategy, between the 'none' and 'full' bounds.

Headline number: labels needed to close 95% of the gap

$$\text{target} = \text{AUROC}_{\text{none}} + 0.95 \cdot (\text{AUROC}_{\text{full}} - \text{AUROC}_{\text{none}}) \tag{13}$$

$$\text{labels needed} = B_{i-1} + (\text{target} - A_{i-1}) \cdot (B_i - B_{i-1}) / (A_i - A_{i-1}) \tag{14}$$

where B_i is the first budget whose mean AUROC A_i reaches the target

where

$B_0 = 0, A_0 = AUROC_{none}$ the curve starts at 'no labels'

A_i AUROC at budget B_i, averaged over seeds

(14) straight-line interpolation between the two budgets around the target; 'not reached' if even 800 labels are not enough

12 · The four experiments

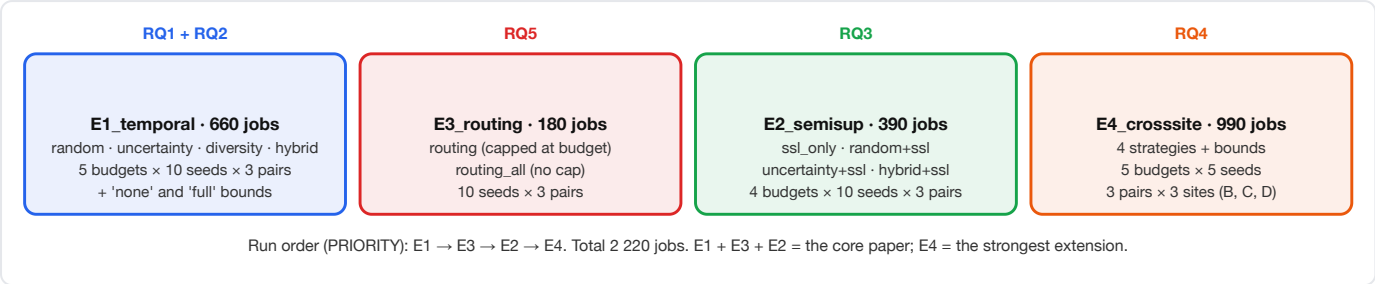


Figure 7. Experiment map (numbers from config.py).

Experiment	Job count, derived
E1_temporal	3 pairs × 10 seeds × (2 bounds + 4 strategies × 5 budgets) = 3 × 10 × 22 = **660**
E3_routing	3 × 10 × (routing_all + routing × 5 budgets) = 3 × 10 × 6 = **180**
E2_semisup	3 × 10 × (ssl_only + 3 strategies × 4 budgets [100, 200, 400, 800]) = 3 × 10 × 13 = **390**
E4_crosssite	3 pairs × 3 sites × 5 seeds × (2 bounds + 4 strategies × 5 budgets [25, 50, 100, 200, 400]) = 45 × 22 = **990**
Total	**2 220 jobs**; a job = (experiment, pair, site, strategy, budget, seed), e.g. *E1_temporal__ecoli_cro__DRIAMS-A__hybrid__b200__s3*

Each job stores, for every round: labels bought, resistant labels bought, number of ambiguous isolates, pseudo-labels and their accuracy, errors on untested isolates, and AUROC / AUPRC / VME / ME / balanced accuracy / threshold on the test set, plus data sizes, device, software versions and run time.

13 · Randomness and the multi-seed design

A single run can be lucky. Each setting is repeated with different **random seeds** (10 for E1–E3, 5 for E4) and results are reported as mean ± 95% confidence interval. What a seed changes, and what it does not:

Changes with the seed	Fixed for all seeds (so comparisons stay fair)
which 20% of 2015–16 is validation (stratified) XGBoost's random row/feature sampling (subsample, colsample) random strategy draws, k-means starts, routing subsets (seeded per seed × round × strategy) cross-site pool/test split (seed + 1000)	test set (all of 2018) the 2017 rounds (dates or hash of code) PCA projection (fitted once, random_state 0) tuned hyper-parameters (one set per pair) the starting model of a pair + seed: identical for every strategy and budget

Paired design. Because all strategies with the same seed start from the same model, validation set and threshold, they can be compared seed by seed ("common random numbers"). That removes most noise from the comparison and is why paired tests are used:

$$\mathrm{CI}_{95} = \mathrm{mean} \pm 1.96 \cdot \mathrm{sd} / \sqrt{\mathrm{n}} \tag{15}$$

$$\mathrm{d_s} = \mathrm{AUROC}_{\mathrm{strategy}}(\mathrm{seed\ s}) - \mathrm{AUROC}_{\mathrm{random}}(\mathrm{seed\ s}) \tag{16}$$

then a Wilcoxon signed-rank test on the 10 differences $\mathrm{d_s}$, with Holm correction across all comparisons

where

sd, n standard deviation over seeds; number of seeds

Wilcoxon signed-rank a test of whether the paired differences are centred above zero, without assuming a bell curve (good for 10 values)

Holm correction many tests are run (strategies × budgets); Holm multiplies the smallest p-value by m, the next by m−1, … so the chance of any false 'significant' result stays ≤ 5%

The confidence interval of "labels needed" (14) comes from a **bootstrap over seeds**: draw 10 seeds with replacement 1 000 times, recompute (14), take the 2.5% and 97.5% values. **Limit:** seeds capture training randomness, not a different 2018. Adding a bootstrap over test isolates is a cheap improvement for the paper.

14 · Ablation study

An **ablation** changes one component and keeps everything else fixed, to show what that component contributes. Some are already built into the job list; others are one-line config changes worth adding.

Already in the 2 220 jobs

Comparison	What it isolates
uncertainty vs random; diversity vs random	value of model doubt / of spectral coverage alone
hybrid vs uncertainty and vs diversity	whether combining both helps (RQ2)
none vs full vs strategies	how much of the decay labels can recover at all (the gap)
budgets 40 → 800 (25 → 400 cross-site)	the shape of the label-efficiency curve
random+ssl vs random; uncertainty+ssl vs uncertainty; ssl_only	value of free pseudo-labels with and without bought labels (RQ3)
routing vs routing_all vs uncertainty	budget cap on safety routing; conformal sets vs plain uncertainty (RQ5)

Suggested extra ablations (not scheduled yet)

Knob in config.py	Values to try	Question answered
MODEL	"xgb" vs "lr"	do the conclusions depend on the learner?
N_ROUNDS	1, 4, 12	does choosing in smaller, more frequent batches help?
NEW_LABEL_WEIGHT / TARGET_LABEL_WEIGHT	1, 2, 5	should fresh labels count more than old data (faster adaptation)?
PCA_DIM	20, 50, 100	is diversity sensitive to the embedding size?
HYBRID_POOL_FACTOR	2, 4, 8	balance between uncertainty and diversity inside hybrid
TARGET_SENSITIVITY	0.90, 0.95	stricter safety threshold: effect on VME and on uncertainty selection
CONFORMAL_ALPHA	0.05, 0.10, 0.20	how many isolates routing sends, and the errors left
SSL_P_HIGH / LOW, SSL_WEIGHT	0.9/0.1, 0.95/0.05, 0.99/0.01; 0.25, 0.5, 1	when do pseudo-labels start to hurt?

Rule for adding ablations: job ids do not encode settings, so never edit config.py while workers are running. Add a *new experiment name* (e.g. "E1_lr") with its own settings instead; old results stay valid.

15 · How the code runs

Notebook	What it does	Run
00 Setup and T4 test	checks Drive access, GPU, runs the 17-check self-test on synthetic data, times one XGBoost fit	once per Colab account
01 Get DRIAMS data	Dryad token (valid 10 h) → peek at archive layout → stream A (and B, C, D) → compact copy	once, one account
02 Data audit and tuning	counts and % R per pair/year, date availability, 16-combination tuning, **premise check** (does a 2015–16 model decay on 2018, and how much does full retraining recover?)	once
03 Worker	picks unfinished jobs, saves after every round, stops before its time budget	any account, many times, in parallel
04 Analyze	tables (mean ± CI, labels needed, Wilcoxon/Holm) and curve figures	any time

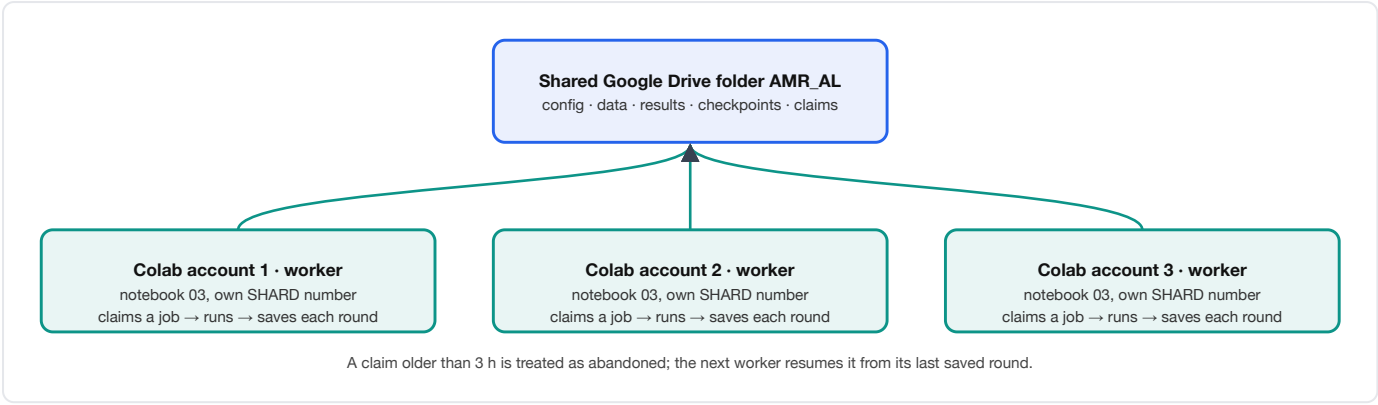


Figure 8. Several free Colab accounts share one Drive folder. Each job's **claim** file prevents double work; **checkpoints** after every round mean a disconnect loses at most one round; a **fingerprint** of the pool/test isolate codes makes sure a resumed job uses the same data.

16 · Risks, and what to check first

Risk	Why it matters	How we find out / respond
The premise: little decay in DRIAMS-A	if a 2015–16 model is almost as good in 2018 as a retrained one, label-efficient retraining has little to recover	notebook 02 prints the "gap" (full – none AUROC) per pair; if < ≈ 0.02, discuss with supervisors and let RQ4 (cross-site, large documented gaps) lead
No acquisition dates	rounds would be pseudo-random, not real quarters	notebook 02 reports date coverage; the paper states which mode was used
Few resistant isolates per round	unstable VME and AUPRC; tiny budgets may contain no R at all	audit table; report R bought per strategy; use per-class conformal
Simulation, not a real lab	reviewers may ask about real-world feasibility	state clearly; it is the standard protocol; a local hospital dataset would lift the paper
Conformal guarantee breaks under drift	routing may miss more R than promised	measure coverage on 2018 and report it (part of RQ5)
Novelty not formally checked	"not done before" must be defensible	run the Scopus and PubMed queries in AMR_Literature_Check.pdf through the FAST library; save date and hit counts
Archive layout differs from assumptions	download code may need adjusting	notebook 01 peeks first and stops if paths differ

Order of work from here

1. Notebook 00 in each account → report "17/17" and GPU timing.
2. Notebook 01 (one account) → peek list and compact data summary.
3. Notebook 02 → audit tables, tuned settings, the three premise "gap" lines → **supervisor check-in**.
4. Notebook 03 workers (E1 first) → notebook 04 tables and figures.
5. In parallel: formal Scopus/PubMed novelty search; confirm target-journal impact factors on JCR.

17 · Key references

Tier: **A** = peer-reviewed · **C** = not peer-reviewed. Read each full text before citing.

Tier	Reference
A	Weis C, Cuénod A, Rieck B, et al. Direct antimicrobial resistance prediction from clinical MALDI-TOF mass spectra using machine learning. Nat Med 28:164–174 (2022). doi:10.1038/s41591-021-01619-9 (DRIAMS)
A	Wiesmann N, Enders D, Westendorf A, Koch R, Schaumburg F. Prediction of AMR from MALDI-TOF mass spectra using machine learning: a validation study. J Clin Microbiol 63:e01186-25 (2025)
A	Duroux et al. Zero-shot MALDI-TOF AMR prediction with masked autoencoders across hospitals and years. GigaScience (2026). doi:10.1093/gigascience/giaf156
A	Corvelo Benz, Miranda, Chen, Sattler, Borgwardt. Conformal prediction for MALDI-TOF AMR (2026). doi:10.1177/15578666251396558
A	Wu Y et al. Iterative discovery of potent polymeric antibiotics via multi-stage and multi-task learning. Nat Commun 17:1878 (2026) (active learning with a wet-lab oracle)
A	Semi-supervised multi-species MALDI-TOF AMR on DRIAMS. Neural Computing and Applications (2026). doi:10.1007/s00521-026-12425-0
A	Chen T, Guestrin C. XGBoost: a scalable tree boosting system. KDD 2016
A	Sener O, Savarese S. Active learning for convolutional neural networks: a core-set approach (k-center greedy). ICLR 2018

A	Angelopoulos A, Bates S. Conformal prediction: a gentle introduction. Foundations and Trends in Machine Learning (2023)
A	Holm S. A simple sequentially rejective multiple test procedure. Scand J Stat (1979); Wilcoxon F. Individual comparisons by ranking methods. Biometrics Bulletin (1945)
C	Settles B. Active learning literature survey. Univ. Wisconsin–Madison technical report 1648 (2009)